Aula 0 — Como se programa um microcontrolador

Seis placas, um chip nu e o mesmo LED

Microcontroladores — DENE/UFMT

OBJETIVOS

- Separar **chip**, **placa** e **ecossistema**, e reconhecer na mesa de demonstração o microcontrolador de cada placa.
- Explicar o que distingue um microcontrolador de um computador, usando a unidade de gerenciamento de memória como fronteira.
- Ler o diagrama de blocos de um microcontrolador como um mapa: nomear cada periférico em uma frase e localizar a aula que o abre.
- Escrever a mesma lógica em Python e em C, e apontar o que desaparece na travessia: a biblioteca, o sistema operacional e o retorno de `main`.
- Descrever os dois caminhos de gravação e explicar por que, neste kit, `#pragma config` não tem efeito.
- Situar o próprio trabalho no semestre: as duas placas, o termostato e o formato dos roteiros.

1 O curso

Meia hora desta aula é regra e calendário. Vale ler antes, porque quase toda reclamação de fim de semestre nasce de algo que estava escrito nesta seção.

1.1 Duas disciplinas, um objeto

Teoria e laboratório são duas disciplinas, com 32 horas e nota cada uma. São também a mesma coisa: o laboratório existe para construir aquilo que a teoria descreve, e a teoria existe porque o laboratório iria falhar sem ela. Você pode ser aprovado em uma e reprovado na outra, mas não vai passar em nenhuma tratando-as como assuntos separados.

O laboratório implementa a teoria da semana anterior, não a do mesmo dia. Os dois primeiros roteiros, R0 e R1, são introdutórios e não valem nota; a partir do R2 o laboratório anda um encontro atrás da teoria. O motivo é dar uma semana de decantação entre ouvir e fazer — e permitir que o miniteste da semana seguinte cobre o que você **fez**, e não só o que ouviu.

1.2 Duas placas

O curso usa dois microcontroladores, e cada um tem um papel.

Placa	Papel	Onde
PIC18F4550, no kit XM118	A máquina que se enxerga inteira: memória, ciclos, pinos e periféricos, um registrador por vez	encontros 0 a 12; R0 a R11
STM32F401, na Nucleo-F401RE	A máquina que resolve o que o PIC não resolve: PWM trifásico, DMA, sistema operacional de tempo real, ponto flutuante, inferência de redes	encontros 9, 13 e 14; S1 e S2

Nenhum conceito é dado duas vezes. O STM32 não repete o que o PIC já ensinou: ele aparece quando o PIC chega a um limite que vocês mediram.

1.3 O que vamos construir

O curso não é uma coleção de experimentos. É **um** artefato, construído em camadas: um termostato digital. Cada roteiro acrescenta um módulo ao mesmo código, escrito por você. Um módulo de referência é publicado depois de cada sessão, para que quem tropeçou numa semana não fique travado na seguinte.

Roteiro	O que passa a existir
R3	Um pino que liga e desliga em tempo controlado
R4	Uma tela que mostra números — o seu driver do display
R5	Um botão que é lido uma vez por aperto, apesar do repique
R6	Uma temperatura lida do mundo real, e a primeira coleta de dados no computador
R7	Um relógio que não depende do laço, um contador de voltas e uma saída proporcional
K1	O esquemático de um pedaço do kit, desenhado no KiCad e conferido com o multímetro
R8	Um programa que atende eventos na hora certa, sem esperar
R9	Uma meia-ponte de potência, acionada pelo PIC num inversor de verdade
R10	Uma decisão automática que não destrói o relé
R11	Uma conversa nos dois sentidos com o computador, e um alvo que sobrevive a desligar a placa
S1	No STM32: telemetria que não trava o processador
S2	No STM32: uma rede neural que reconhece a ventoinha com defeito

Ao final, o objeto na sua bancada é o mesmo que existe dentro de uma estufa, de uma incubadora ou de um ar-condicionado inverter. Não é um modelo reduzido dele: é ele.

1.4 Calendário

#	Teoria	Laboratório	Miniteste
0	Como se programa um microcontrolador	R0 — C, do PC ao micro	—
1	Memória e folha de dados	R1 — o mapa das chaves, com multímetro	M1
2	O relógio e o ciclo de busca-execução	R2 — classe de memória em Python	M2
3	Barramento paralelo e o HD44780	R3 — pino, tempo e osciloscópio	M3
4	Entrada digital: o pino como sensor	R4 — o seu driver do display	M4
5	Aquisição analógica: ADC e LM35	R5 — botões e repique	M5
6	Temporizadores e PWM	R6 — temperatura no display e primeira coleta	M6
7	**Avaliação integradora I**	R7 — tique, tacômetro e PWM no cooler	—
8	Interrupções	K1 — do kit ao esquemático, no KiCad	M8
9	Estágio de potência: do ECCP do PIC ao TIM1 do STM32	R8 — interrupções	M9
10	Controle liga-desliga e histerese	R9 — PWM e meia-ponte no inversor	M10
11	Comunicação serial: UART e I²C	R10 — termostato liga-desliga	M11
12	**Avaliação integradora II**	R11 — serial nos dois sentidos e EEPROM	—
13	Arquitetura de firmware e RTOS (STM32)	Folga — fechamento do termostato	M13
14	Controle e IA embarcada (STM32)	S1 — STM32: telemetria sem bloquear	M14
15	**Seminários comparativos** e encerramento	S2 — STM32: vibração e IA	—

Não há prova de laboratório. O que se avalia na bancada é o que você faz em cada roteiro, toda semana. O encontro 13 do laboratório é a folga do semestre: se nada atrasar, é a sessão em que o termostato fecha com os seus próprios módulos.

1.5 Como se avalia

Os minitestes ocupam os dez minutos finais da aula de teoria e cobram o que foi visto naquele dia e nos anteriores. Nunca cobram o que ainda não foi coberto: se uma questão parece pedir conteúdo futuro, é erro meu, e vale reclamar na hora.

Teoria: instrumento	Peso
Minitestes (contam os 8 melhores)	20%

Avaliação integradora I	25%
Avaliação integradora II	30%
Seminário comparativo	25%

Laboratório: parte de cada roteiro	**Pontos**
Previsões, antes da sessão	2,0
Função do pré-lab, testada no simulador	2,0
Tarefas de bancada	4,0
Defesa de uma linha do seu código	2,0

A nota de laboratório é a média dos roteiros R2 a R11, do K1, do S1 e do S2, sem descarte. R0 e R1 não valem nota.

CONCEITO

Todo roteiro tem código para você escrever. Antes da sessão você recebe o projeto montado com **uma função vazia**, escreve essa função e a testa no simulador. Na bancada você grava, mede, e a medida confirma ou derruba a sua função. O tempo da sessão é para medir e corrigir, não para começar.

No fim, o professor ou o monitor aponta uma linha do **seu** código e pergunta por que ela está ali. Um aluno do grupo é sorteado para responder. Não há resposta decorada para essa pergunta: ou você escreveu a linha sabendo o que ela faz, ou não escreveu.

No seminário comparativo, a nota do grupo é ajustada por arguição individual. Isso não é desconfiança: é a única forma honesta de diferenciar quem estudou a plataforma de quem estudou os slides do colega na véspera.

1.6 Regras de convivência

- O material de cada encontro é publicado na página do curso **antes** da aula. Vir com ele lido muda a aula de ditado para discussão.
- O módulo de referência de cada roteiro é publicado **depois** da sessão. Use-o para conferir, não para começar.
- Roteiro sem previsão preenchida não é corrigido. A previsão vale pelo raciocínio, não pelo acerto — errar uma previsão bem argumentada não custa nota.
- A bancada é em grupo. O pré-lab de código e as previsões são individuais.

ATENÇÃO

Segurança de bancada, antes de qualquer coisa: a **garra de terra do osciloscópio vai num ponto de GND, e em nenhum outro lugar.** Presa num ponto de 12 V — LAMP, HEATER, COOLER — ela curto-circuita esses 12 V contra o terra da rede.

Cumprida essa regra, medir é seguro: com a garra no GND, a ponteira pode ir a qualquer ponto do circuito. Este curso é feito de medições, e a regra existe para que vocês possam medir à vontade, não para que evitem o instrumento.

O tratamento completo, com a placa na mão, está no R1. Aqui fica o aviso mínimo, porque alguns de vocês vão encostar num osciloscópio antes disso.

2 O que é um microcontrolador

2.1 A mesa

EXPERIMENTO

Na mesa há sete objetos rodando **o mesmo programa** — um LED que pisca:

1. Um PIC18F4550 espetado numa protoboard, com um capacitor, um resistor e um LED. Nada mais.
2. O kit XM118, com display, relés, sensor de temperatura e quarenta outras coisas soldadas em volta de um PIC18F4550 idêntico.

3. Um Arduino Uno.
4. Uma Raspberry Pi Pico.
5. Uma placa com ESP32.
6. Uma Nucleo-F401RE, da STMicroelectronics.
7. Uma plaquinha com um CH32V003, um microcontrolador que custa centavos de dólar.

Antes de continuar lendo, responda: **qual deles pisca mais rápido?**

Todos piscam igual. O atraso de cada um foi programado para dar o mesmo período, e o LED não tem como mostrar mais nada. O que distingue essas placas não se vê piscando um LED — e é disso que trata o resto do semestre.

2.2 Chip, placa e ecossistema

Quase sempre que alguém diz "programei um Arduino", três coisas diferentes estão misturadas na frase.

Placa	Microcontrolador	Núcleo	O que se destaca
Arduino Uno	ATmega328P	AVR, 8 bits	O ecossistema mais conhecido
XM118	PIC18F4550	PIC18, 8 bits	USB no próprio chip; o micro deste curso
—	CH32V003	RISC-V, 32 bits	Custa uma fração de qualquer outro da mesa
Pico	RP2040 ou RP2350	Cortex-M0+ ou M33	O RP2350 tem também núcleos RISC-V
ESP32 DevKit	ESP32	Xtensa, 32 bits, dois núcleos	Wi-Fi e Bluetooth no chip
Nucleo-F401RE	STM32F401	Cortex-M4 com FPU	O segundo micro deste curso

CONCEITO

O chip é o microcontrolador: um componente de silício. **A placa** é o circuito em volta dele — conectores, LEDs, reguladores, um gravador embutido. **O ecossistema** é o software que alguém escreveu para você: o ambiente de programação, as bibliotecas, os exemplos, a comunidade.

Arduino é placa e ecossistema; o microcontrolador é o ATmega328P. A Nucleo é placa; o microcontrolador é o STM32F401. A XM118 é placa; o microcontrolador é o PIC18F4550.

Este curso vai manter essas três camadas separadas o tempo todo — e acrescentar uma quarta, o simulador. Muito do que dá errado na bancada é atribuir a uma camada o que é de outra.

NOTA

Se o CH32V003 custa uma fração do PIC e tem memória parecida, por que o curso usa PIC e STM32? Porque o PIC é pequeno o bastante para ser entendido inteiro, e tem um kit com quarenta periféricos pronto na bancada. O STM32 entra quando o problema fica grande demais para o PIC. Preço é critério de produto, não de aula — e no seminário vocês vão ter de defender a escolha de uma plataforma para um produto.

2.3 O que o chip nu precisa

Elemento	Por que é obrigatório	O que acontece sem ele
V_{DD} e V_{SS} (dois pares)	Alimentação do núcleo e das portas	Não liga, ou liga de forma instável — os dois pares precisam estar conectados
Capacitor de desacoplamento (100 nF)	Fornece corrente nas transições rápidas que a fonte não acompanha	Funciona na bancada e falha aleatoriamente depois; o defeito mais difícil de achar da lista
MCLR em nível alto	MCLR baixo mantém o processador em reset	Fica travado em reset para sempre, sem nenhum sintoma visível

Fonte de sinal de relógio	Sem relógio não há ciclo de instrução	Nada executa. O 4550 tem oscilador interno, então este item é opcional — e é por isso que a protoboard não tem cristal
Resistor em série com o LED	Limita a corrente do pino	O pino entrega corrente até se danificar

CONCEITO

Note o que **não** está na lista: memória, barramento, conversor A/D, temporizador, porta serial. Não estão porque já estão dentro do encapsulamento. Essa é a definição operacional de microcontrolador — o computador inteiro num único componente, com periféricos junto.

O que está na lista é apenas o que o silício não consegue fabricar dentro de si: energia, uma referência de tempo, e a garantia de que ninguém o está segurando em reset.

2.4 O que o kit acrescenta

Dentro do chip	Fora, na placa
Memória de programa, RAM, EEPROM interna	EEPROM I²C externa
Conversor A/D, comparadores	Sensor LM35, potenciômetros
Temporizadores, PWM	Ventilador, lâmpada, aquecedor
Módulo USB (transceptor incluído)	Conector USB e proteção
Portas digitais	LEDs com resistores, chaves, teclado matricial, display, relés

CONCEITO

Nada no seu programa diz que RD0 é um LED. Essa informação não está no `xc.h`, não está na folha de dados do PIC18F4550, não está no compilador. Ela está numa chave de duas posições no painel da placa.

Toda camada que o kit acrescenta é uma camada que nenhuma ferramenta conhece — e é onde vão morar quase todos os defeitos deste semestre. O R1 começa exatamente por aí: o mapa das chaves, medido com o multímetro.

2.5 A fronteira: a unidade de gerenciamento de memória

Vocês já escreveram C. O mesmo compilador, apontado para outro destino, produz algo com três diferenças que aparecem já no primeiro programa.

No computador	Aqui	Consequência no código
Existe sistema operacional	Não existe	Não há para onde retornar. `main` não termina: termina num laço infinito, ou o processador continua buscando instruções no que vier depois
Existe `printf`	Não existe	A saída é um pino. Escrever num pino é escrever numa posição de memória
O programa é carregado na RAM	O programa mora na Flash	Variáveis inicializadas precisam ser copiadas da Flash para a RAM antes de `main` — por um código que ninguém escreveu

A tabela sugere uma fronteira nítida que hoje não existe. A Pico da mesa tem mais RAM que os primeiros PCs; o ESP32 roda pilha TCP/IP e sistema de arquivos. Contar transistores, megahertz ou quilobytes não separa mais as duas coisas.

CONCEITO

A diferença que ainda separa é a **unidade de gerenciamento de memória**, a MMU. Ela traduz endereços: o endereço que o programa usa não é o endereço físico. É isso que permite processos isolados, memória virtual e um sistema operacional que sobrevive a um programa mal escrito.

Um microcontrolador não tem MMU. **O endereço que o programa escreve é o endereço que existe no chip**, e não há ninguém entre os dois. Um ponteiro errado corrompe qualquer coisa, inclusive o que você não queria tocar.

O máximo que um microcontrolador maior oferece é uma **unidade de proteção de memória**, a MPU — o STM32 da mesa tem uma. Ela proíbe o acesso a certas regiões, mas não traduz endereço nenhum.

NOTA

É por isso que um Cortex-A, o processador de um celular, roda Linux, e um Cortex-M, o da Nucleo, não roda. E é por isso que, no encontro 2, o simulador do PIC consegue tratar a memória como uma lista de Python: o endereço é o índice, e não há tradução nenhuma no meio.

3 Os periféricos: o mapa do curso

A folha de dados do PIC18F4550 traz um diagrama de blocos do chip: o núcleo no centro, a memória ao lado, e em volta dele os periféricos ligados às portas. Projetado na tela, esse diagrama é o mapa do semestre. Cada bloco recebe uma frase hoje e uma aula depois.

Bloco do PIC18F4550	O que faz, em uma frase	Aula
Portas de entrada e saída	O pino é um bit num registrador	2 e 4
Conversor A/D	Transforma tensão em número	5
Temporizadores	Contam tempo ou eventos sem o processador	6
CCP e ECCP	Geram PWM e medem período	6 e 9
Interrupções	O hardware chama o seu código	8
Comparadores	Comparam duas tensões sem o conversor	10
EUSART	Comunicação serial ponto a ponto	11
MSSP	Barramento I²C de dois fios, com endereço	11
EEPROM de dados	Memória que não apaga ao desligar	11
USB	O que o bootloader usa toda semana	hoje e 11

CONCEITO

Os mesmos blocos existem no STM32 da mesa, em maior número e com um a mais que muda tudo: o **DMA**, que copia dados entre periférico e memória sem o processador. O ESP32 acrescenta o rádio. A Pico acrescenta o PIO, pequenos processadores dedicados a inventar periféricos.

O que muda de um microcontrolador para outro da mesa não é a lista de ideias. É **quais** blocos existem e **de que tamanho** eles são. Quem entende os blocos do PIC lê o diagrama de qualquer um deles.

4 As camadas, e por que descemos todas

Boa parte de vocês já piscou um LED. Em Arduino, em ESP32, ou num STM32 pelo configurador gráfico. E a pergunta legítima, na primeira aula, é por que este curso vai fazer de um jeito mais trabalhoso.

A resposta não é que aquilo é para amadores. Não é: Arduino roda dentro de produto que se vende. A resposta é que existem camadas entre o seu `if` e o elétron, e um curso de microcontroladores é aquele em que se desce por todas.

Camada	O que ela oferece	Quem a escreveu
Seu programa	A lógica que você quer	Você
Framework ou HAL	**Comportamento**: acender um pino, ler um valor	Alguém que nunca viu a sua placa
Mapa de registradores	**Nomes**: `LATD` significa 0xF8C	O fabricante do chip
Endereço cru	Nada. Um número e um ponteiro	Você, se nada existisse
Registrador	Um endereço com significado elétrico	O projetista do silício
Pino	Um transistor ligado a um fio	A física

4.1 O mesmo pisca, sem nenhuma biblioteca

CONCEITO

A camada do "endereço cru" merece ser vista uma vez. Sem nenhum arquivo de apoio, escrever num pino do PIC18F4550 é isto:

```
#define LATD  (*(volatile unsigned char *) 0x0F8C)

LATD = 0x01;
```

E no ATmega328P do Arduino Uno, sem a biblioteca do Arduino, exatamente a mesma forma:

```
#define PORTB (*(volatile unsigned char *) 0x25)
```

Isso não é AVR contra PIC. É a mesma linha nas duas plataformas. O que faz `LATDbits.LATD0 = 1` parecer amigável não é o chip: é o `xc.h` conter alguns milhares de linhas assim, escritas por outra pessoa.

O pisca do Arduino Uno escrito direto nos registradores, sem `digitalWrite`, fica muito próximo do pisca do PIC. As diferenças são de convenção, não de ideia.

	PIC18F4550 (XM118)	ATmega328P (Arduino Uno)
Valor do pino	`LATD`, em 0xF8C	`PORTB`, em 0x25
Direção	`TRISD`: **0** é saída	`DDRB`: **1** é saída
Instrução de bit	`BCF` e `BSF`	`CBI` e `SBI`
Atraso pronto	`__delay_ms`, uma macro	`_delay_ms`, uma macro
Gravação	bootloader pela USB	bootloader pela serial da USB
Arquivo gravado	Intel HEX	Intel HEX, o mesmo formato

NOTA

A polaridade da direção é o tipo de detalhe que pega quem copia código de uma família para outra: a mesma ideia, com o bit invertido. O compilador não avisa, porque para ele os dois são só bytes.

A Raspberry Pi Pico acrescenta uma ideia que nenhum dos dois tem: registradores separados para **ligar**, **desligar** e **inverter** bits, que mexem só nos bits pedidos sem ler o resto da porta. O motivo de eles existirem aparece no encontro 2, no problema de ler, modificar e escrever.

PREVISÃO

Os dois rodam a 16 MHz — o PIC deste kit e o ATmega do Uno. Com o mesmo laço de atraso, contando até o mesmo número, **qual dos dois pisca mais rápido, e por quanto?**

Registre o palpite. A resposta é a primeira conta do encontro 2.

ATENÇÃO

Não é para você entender a linha do `#define` hoje, e vale dizer por quê. Ela empilha seis ideias, e nenhuma delas existe em Python:

um **endereço** como número; a **conversão** desse número para ponteiro; a **desreferência** do ponteiro; a palavra `volatile`, que proíbe o compilador de otimizar o acesso; a base **hexadecimal**; e a ideia de que um endereço pode **ser** um periférico, em vez de guardar um valor.

O curso desce essas seis, uma por vez. No encontro 1 esta linha fica legível, e no encontro 2 você escreve o outro lado dela — o pedaço de memória que responde quando alguém escreve em 0x0F8C.

`xc.h` não é uma HAL. Ele é a camada de baixo da tabela das camadas: dá nomes, não comportamento. CONCEITO

A diferença é observável. `digitalWrite(13, HIGH)` funciona em qualquer pino de qualquer placa — é comportamento, e é portátil. `LATDbits.LATD0 = 1` só significa alguma coisa neste chip, neste pino. O código com HAL viaja; o nosso não.

E é exatamente por isso que um custa e o outro não.

4.2 O mesmo pisca, em três camadas

Como se escreve	Custo	O que está escondido
`digitalWrite(13, HIGH)`	dezenas de ciclos	Tabela de pinos, desligamento de PWM, interrupções, leitura-modificação-escrita
`IO_RD0_SetHigh()`	**um ciclo**	Nada: é uma macro que vira a linha de baixo
`LATDbits.LATD0 = 1`	**um ciclo**	Nada

Abstração em tempo de compilação é grátis; em tempo de execução, se paga em ciclos. CONCEITO

A macro do configurador da Microchip desaparece na compilação: sobra a mesma instrução única. A função do Arduino existe em tempo de execução — ela consulta uma tabela para descobrir qual porta corresponde ao pino 13, verifica se há um temporizador de PWM ativo naquele pino e o desliga, protege a operação contra interrupção, e só então escreve.

Nada disso é desperdício: é o preço de a mesma linha funcionar em cem placas diferentes. Mas é preço, e saber quando ele importa é conteúdo de engenharia.

4.3 Três razões para descer

Primeira: alguém escreve a abstração. `digitalWrite` é código C que alguém escreveu, e que alguém mantém. Um curso de engenharia forma quem escreve, não apenas quem consome — e para escrever a camada é preciso conhecer a de baixo. CONCEITO

Segunda: a abstração vaza exatamente quando importa. Ela esconde detalhes até o dia em que um deles decide o resultado, e nesse dia não há como consertar por cima. Alguns exemplos que este semestre vai encontrar de frente: o tempo de aquisição do conversor decide se um divisor resistivo pode ser lido (encontro 5); um contador de milissegundos de dezesseis bits dá a volta e o programa mede o tempo errado (encontro 8); uma função de envio serial bloquear ou enfileirar decide se o controle funciona (encontro 11).

Terceira: aqui os recursos não sobram. São 32 kB de programa e 2 kB de dados. Uma abstração que reserva um buffer por canal é uma decisão de projeto, não um detalhe de implementação.

ATENÇÃO

O que este curso não está dizendo. Não está dizendo para não usar biblioteca.

A posição, que vale para o semestre inteiro, é outra: **derive uma vez, e depois use para sempre.** No encontro 11 vocês vão calcular à mão o divisor de uma taxa de comunicação — para que, quando o configurador mostrar uma caixa escrita "9600", vocês saibam que ele programou 9592,3, por que programou isso, e se o erro cabe. No STM32 dos encontros 13 e 14 vocês vão usar a biblioteca do fabricante, e vão saber o que ela esconde.

Quem nunca fez a conta não consegue julgar a caixa. E quem não consegue julgar a caixa não escolheu a biblioteca: aceitou.

5 O mesmo programa em três formas

O escopo de hoje é um LED que pisca. Nada além disso. O ponto não é a dificuldade do programa: é o que muda entre as três versões.

5.1 Em Python

```python
import time

while True:
    print("aceso")
    time.sleep(0.5)
    print("apagado")
    time.sleep(0.5)
```

Duas coisas aqui são serviços que alguém prestou a você. `print` supõe um terminal, um sistema operacional e uma biblioteca. `time.sleep` supõe um relógio mantido por esse mesmo sistema, e que existe outra coisa para o processador fazer enquanto você espera.

NOTA

Python também roda **dentro** de um microcontrolador: a Pico da mesa sai de fábrica pronta para o MicroPython. Mas repare no que isso significa — o interpretador de Python é um programa em C, gravado na Flash, que lê o seu texto e o executa. Os serviços continuam existindo; só passaram a morar no chip, e a custar memória e ciclos dele.

5.2 Em C, no computador

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    while (1) {
        printf("aceso\n");   usleep(500000);
        printf("apagado\n"); usleep(500000);
    }
    return 0;
}
```

Mudou a sintaxe. Não mudou nenhuma das duas suposições.

5.3 Em C, no microcontrolador

```c
#include <xc.h>

void main(void)
{
    unsigned int i, j;

    LATD  = 0x00;          // primeiro decide o valor
    TRISD = 0x00;          // depois deixa o pino sair
```

```
    while (1) {
        LATDbits.LATD0 = 1;
        for (j = 0; j < 10; j++) for (i = 0; i < 30000; i++);
        LATDbits.LATD0 = 0;
        for (j = 0; j < 10; j++) for (i = 0; i < 30000; i++);
    }
}
```

Era	Virou	Porque
`printf`	`LATDbits.LATD0`	A saída é um pino, e o pino é um nome que o compilador traduz para um endereço de memória
`usleep`	um laço contado	Não há sistema operacional para acordar você; a única forma de esperar é gastar instruções
`return 0`	`while (1)`	Não existe destino para o retorno

CONCEITO

`LATD` não é uma variável do seu programa. É um nome para uma posição fixa da memória, e aquela posição está fisicamente ligada aos pinos. A atribuição `LATDbits.LATD0 = 1` é uma escrita em memória como qualquer outra — só que essa escrita move elétrons num fio que sai do encapsulamento.

Este é o fio condutor do primeiro terço do curso. No encontro 1 vamos descobrir **qual** endereço. No encontro 2, vamos escrever nele à mão.

ATENÇÃO

`LAT` antes de `TRIS`. Por enquanto, decore: primeiro você escreve o valor que o pino deve ter, depois libera o pino para sair. A ordem inversa faz o pino assumir por alguns instantes o valor que estava lá antes — o que é irrelevante com um LED e é sério com um relé de 12 V ligado a um aquecedor.

No encontro 2 isso deixa de ser regra decorada e passa a ser consequência.

NOTA

No XM118, os LEDs **acendem** quando a chave do painel é fechada, com o processador ainda em reset e sem que uma linha de C tenha executado. Portanto o evento observável do seu primeiro programa é **apagar**, não acender. Se você gravar esperando ver um LED acender, vai concluir que não funcionou.

5.4 Um atraso cuja duração você não sabe

Repare no que acabamos de fazer: escrevemos um programa cujo período ninguém nesta sala sabe calcular. Isso é deliberado.

Para saber o período faltam dois números: quantos ciclos de máquina custa uma iteração do laço, e quanto tempo dura um ciclo. Nenhum dos dois está no código, e nenhum dos dois vamos obter hoje.

Repare também que o laço é **duplo**. Um contador de 16 bits não chega perto de um pisca visível nesta plataforma, e você não tem como saber disso antes de tentar. Na bancada você vai ajustar os dois valores por tentativa até enxergar o LED comutar — que é a única técnica disponível para quem não sabe contar ciclos. Anote o par que funcionou.

NOTA

A dívida. No encontro 2, com a listagem gerada pelo compilador na mão e o valor de τ_{cy}, você calcula o período que hoje só chutou. No R3, com o osciloscópio, você mede e descobre de quanto errou.

Anote o palpite de hoje. Sem ele, o encontro 2 vira exercício de aritmética; com ele, vira conferência.

Existe uma macro `__delay_ms` no XC8 que parece resolver isso. Não vamos usá-la hoje. Ela exige uma constante conhecida em tempo de compilação e depende de um símbolo

que você precisa definir com o valor certo — e explicar por quê consome quinze minutos de um assunto que não é o de hoje. É a primeira seção do encontro 2.

NOTA

Um laço vazio pode ser removido pelo compilador: ele não produz efeito observável, e o compilador tem licença para descartá-lo. Nas nossas condições ele sobrevive, mas o sintoma de quando não sobrevive é bonito e vale reconhecer — o LED não pisca, e sim fica com meio brilho. No encontro 2, ao ler a listagem, você vai poder verificar se o laço ainda está lá.

6 Como o código entra no chip

6.1 Do arquivo ao conteúdo da Flash

```
main.c → [pré-processador] → [compilador] → [linker + biblioteca] → main.hex
```

O `.hex` não é binário: é texto. Cada linha diz quantos bytes vêm, em qual endereço eles devem ser escritos, de que tipo é o registro, os bytes em si, e um byte de verificação. Abrir esse arquivo num editor de texto é uma das coisas que faremos no R0 — junto com as representações binária e hexadecimal, que existem porque são os dígitos dos endereços.

6.2 Dois caminhos

Você vai usar o caminho fácil o semestre inteiro. É justo saber que existe outro, e o que o fácil esconde.

	Bootloader USB	**Gravador (MPLAB Snap)**
Como	Segurar SW9 e ligar; um aplicativo no PC envia o `.hex` pela USB	Cinco fios em MCLR/V_{PP}, PGC, PGD, V_{DD}, V_{SS}
Hardware extra	Nenhum	O gravador
Exige	Que o bootloader já esteja gravado no chip	Nada; funciona em chip virgem
Grava os bits de configuração	**Não**	Sim
Ocupa memória	Sim, o começo da Flash	Não

CONCEITO

O bootloader não é mágica. É um programa comum, gravado na Flash, que roda antes do seu e decide entre entregar o controle a você ou ficar escutando a USB. Quem escreve na Flash do PIC é o próprio PIC.

Isso também explica por que o chip nu da protoboard exigiu o gravador: nele não há ninguém escutando.

6.3 Os bits de configuração

Além da memória de programa, o PIC18F4550 tem uma pequena área de **bits de configuração**, a partir do endereço 0x300000. Eles decidem coisas que precisam estar resolvidas antes de a primeira instrução executar, e que o programa não muda enquanto roda:

- qual oscilador gera o relógio, e por quanto ele é dividido ou multiplicado;
- se o cão de guarda (watchdog) começa ligado;
- se o pino 1 é o reset ou um pino comum;
- se RB0 a RB4 nascem analógicos ou digitais;
- se o pino RB5 serve à gravação em baixa tensão.

Em C, eles se escrevem com `#pragma config`. É o que o chip nu da protoboard precisou ter no programa da demonstração:

```
#pragma config FOSC   = INTOSCIO_EC  // oscilador interno; RA6 livre
#pragma config WDT    = OFF          // sem cão de guarda
#pragma config MCLRE  = ON           // o pino 1 é o reset
#pragma config PBADEN = OFF          // RB0 a RB4 nascem digitais
#pragma config LVP    = OFF          // RB5 volta a ser um pino comum

void main(void)
{
    OSCCONbits.IRCF = 0b111;         // 8 MHz; o reset deixa em 1 MHz
    /* ... o mesmo pisca de antes ... */
}
```

CONCEITO

Quem grava os bits de configuração é o gravador, junto com o programa. O compilador apenas coloca os valores do `#pragma config` no `.hex`, em registros com endereço 0x300000 em diante.

O bootloader do kit **não grava** essa área. Ele já a gravou uma vez, de fábrica, e não a reescreve. Por isso, na XM118, `#pragma config` no seu código é **silenciosamente ignorado**: compila, grava, não dá erro, não dá aviso e não tem efeito.

No R0 você vai provar isso na bancada, e vai ler de dentro do seu próprio programa os bits que o bootloader deixou.

6.4 O código que veio antes do seu

Vale parar aqui, porque o que acabamos de descrever não é uma peculiaridade desta placa. É a situação normal de todo sistema computacional que você vai encontrar na vida.

CONCEITO

Seu programa quase nunca é o primeiro a rodar. Há código que executou antes, que decidiu entregar o controle a você, e que continua sendo dono de alguns recursos. Você não programa numa máquina vazia: programa no espaço que sobrou.

Onde	Quem rodou antes e o que ele guardou para si
XM118	Bootloader HID: o começo da Flash, os bits de configuração e os endereços de desvio
Nucleo	Bootloader em ROM do STM32 e o gravador embutido na placa: a decisão de qual imagem carregar
Computador	Firmware UEFI: a inicialização do hardware, tabelas de memória e a ordem de carga
Celular	Uma cadeia de bootloaders, cada um verificando a assinatura do seguinte

A consequência prática é sempre a mesma: **existem coisas que o seu programa não pode mudar, porque não são dele.** Descobrir quais são, e por quê, é parte de conhecer qualquer plataforma.

Neste curso essa restrição vai aparecer várias vezes, sempre com a mesma assinatura — uma linha de código que parece correta, não dá erro nenhum, e não tem efeito. E com uma segunda assinatura, mais sutil: o bootloader rodou antes de você e **deixou registradores fora do estado de reset**. O que a folha de dados diz sobre o chip recém-ligado não vale, sem conferir, para o chip que acabou de sair do bootloader.

NESTE KIT

O que o bootloader do XM118 decidiu por você:

a frequência do núcleo é **16 MHz**, com 48 MHz para a USB, e você não escolhe — o encontro 2 mostra como os divisores chegam a esses números;

`ADCON1` chega ao seu programa valendo 0x07, e por isso RB0 a RB4 já começam digitais, ao contrário do que a folha de dados diz para o chip recém-ligado (encontro 4);

o cão de guarda pode estar fora do seu alcance (encontro 6);

os endereços de desvio de interrupção pertencem a ele e precisam ser reencaminhados (encontro 8).

Nada disso vale para o chip nu da protoboard, e nada disso é conteúdo sobre microcontroladores: é conteúdo sobre **esta** placa. O que é conteúdo é o princípio acima.

NESTE KIT

É **SW9** que entra em modo bootloader, não SW1. O manual da Exsto diz SW1 e está errado; a serigrafia da placa tem precedência sobre o manual.

ATENÇÃO

Gravar a XM118 com o gravador, e não pelo bootloader, apaga a Flash inteira — e leva o bootloader junto. A partir daí o kit só volta a ser gravado pela USB depois que alguém regravar o bootloader. No laboratório, o gravador não se liga na XM118.

7 O fio condutor

Os quatro primeiros encontros abrem, um por vez, as camadas de um simulador do PIC18 escrito em Python:

usa o simulador (0) → abre a memória dele (1) → abre a execução (2) → confronta com um periférico real (3)

Cada camada aberta é imediatamente confrontada com a bancada. O simulador é determinístico e mostra o que o osciloscópio mostra rápido demais; a bancada mostra o que o simulador mente. Saber **onde** o simulador mente é conteúdo deste curso, não uma nota de rodapé.

SEM NOTA

R0 e R1 não valem nota. Existem para você errar antes de qualquer avaliação, e para absorver a folga das duas primeiras semanas de matrícula. Aproveite: gravar código errado num microcontrolador é gratuito e reversível, e essa é uma das poucas semanas do semestre em que também é indolor.

8 Previsão para o R0

PREVISÃO

Preencha antes da bancada. Vale o raciocínio, não o acerto.

P1. Com o kit ligado e a chave do painel de LEDs fechada, mas antes de gravar qualquer programa: os LEDs estarão acesos ou apagados? Justifique pelo estado dos pinos em reset.

P2. Seu programa usa dois laços contados encaixados como atraso. Estime o período do pisca em segundos e registre a base do palpite — antes de ajustar qualquer constante na bancada.

P3. O que você espera observar se trocar a ordem das duas linhas de inicialização — `TRISD` antes de `LATD`?

P4. Você acrescenta ao programa do pisca a linha `#pragma config CPUDIV = OSC1_PLL2`, que pede ao chip um relógio três vezes mais rápido. Grava pelo bootloader. O que acontece com o pisca?

P5. Você grava e nada muda no painel. Liste, em ordem, as três primeiras coisas que verificaria — e diga o que cada verificação eliminaria.

BANCADA

Leve para a bancada: esta folha preenchida e o palpite de período do P2 anotado com o número, não "uns dois segundos". No encontro 2 você vai comparar esse número com uma conta, e no R3 com uma medida.

9 Exercícios

TAREFA

Exercício 0.1. Um colega afirma que o programa "acaba" quando `main` retorna.

(a) O que de fato acontece quando `main` retorna num programa de computador?

(b) E no microcontrolador, o que o processador faz depois da última instrução de `main`?

(c) Por que o `while (1)` não é questão de estilo?

Exercício 0.2. Sobre o atraso por laço contado, sem usar nenhum valor numérico de tempo: TAREFA

(a) Se a constante do laço externo dobrar, o que se pode afirmar sobre o período?

(b) Qual é a suposição por trás dessa afirmação?

(c) Por que não se pode afirmar o valor do período?

(d) Se o compilador removesse o laço, qual seria o sintoma observável no LED?

Exercício 0.3. Você compra um PIC18F4550 novo e monta na protoboard. TAREFA

(a) Existe equivalente ao SW9 nessa montagem?

(b) Que caminho de gravação serve, e o que ele exige fisicamente?

(c) O programa dessa montagem precisa de `#pragma config`? E o da XM118?

(d) Depois de gravar o chip da XM118 com um gravador, o caminho fácil continua disponível?

Exercício 0.4. Classifique cada item em **dentro do encapsulamento** ou **fora, na placa**: LED; conversor A/D; resistor de 330 Ω; LM35; temporizador; relé; EEPROM de 256 bytes; display; cristal de 20 MHz; conector USB. Justifique os dois casos que você achar ambíguos. TAREFA

Exercício 0.5. Separe os itens em **chip** e **placa**, e diga o que o **ecossistema** Arduino acrescenta a um deles: Arduino Uno; ATmega328P; Nucleo-F401RE; STM32F401; Raspberry Pi Pico; RP2040; XM118; PIC18F4550. TAREFA

Exercício 0.6. Um colega diz: "esse micro roda a 16 MHz, meu notebook roda a 3 GHz. É duzentas vezes pior." Aponte três coisas que a comparação ignora, e diga em qual delas o micro é **melhor**, não apenas mais barato. TAREFA

No encontro 1: memória e folha de dados. A pergunta que abre a aula é uma conta que não fecha — o endereço de dados tem doze bits, e a instrução carrega oito. De onde saem os quatro que faltam? A resposta a essa pergunta é o mapa de memória inteiro, e é o que você vai implementar em Python no R2. NOTA